

dev-mcp

AI Engineering Operating System

User Manual & Complete Reference Guide

Version	v0.1.0
Author / Maintainer	Pranjal Yadav
Protocol	Model Context Protocol (MCP)
Server Script	server.py
Audience	Beginner – Developer Users

Supported Clients: Cursor | Claude Desktop | Windsurf | Cline / Roo Code | Antigravity CLI

Table of Contents

1. Introduction & What is dev-mcp?
2. Prerequisites & Installation
3. Connecting Your AI Client
4. The Efficiency Workflow Loop
5. Architecture & Technology Rules
6. Reconnecting After Updates
7. Testing Your Server
8. Complete Tool Reference (70 Tools)

1. Introduction & What is dev-mcp?

dev-mcp is an **AI Engineering Operating System** built on the **Model Context Protocol (MCP)**. It acts as a structured brain that connects your AI assistant to your actual project, giving it real-time awareness of your codebase, tasks, phases, and decisions.

Without dev-mcp your AI assistant starts each conversation from scratch with no memory. With dev-mcp it knows your architecture, sprint goals, task backlog, and last session handoff instantly.

What Problems Does It Solve?

- AI assistants forget context between sessions — dev-mcp provides persistent project memory.
- Developers lose track of tasks, phases, and decisions — dev-mcp manages them as structured data.
- Code drift and architecture violations go undetected — dev-mcp audits imports and rules.
- Git commits are messy — dev-mcp generates conventional commit messages automatically.
- Onboarding to legacy codebases is slow — dev-mcp scans and maps existing projects automatically.

How Does It Work?

dev-mcp runs as a local MCP server (a Python subprocess). Your AI client connects to it via a JSON configuration. When you ask your AI to start a session or check the next task, it calls dev-mcp tools behind the scenes — reading and writing structured context files in your project directory.

2. Prerequisites & Installation

2.1 System Requirements

- Python 3.10 or higher (required runtime)
- Git (required for commit, sync, and branch tools)
- pip or uv (Python package manager)
- Virtual environment tool: venv or uv venv
- Primary dependency: fastmcp 2.0.0+ (auto-installed via `pip install -e .`)

2.2 Installing Python & Git

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install python3.10 python3.10-venv python3-pip git -y
```

macOS

```
# Install Homebrew if not present:
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install python@3.10 git
```

Windows

```
# Download Python 3.10+ from https://python.org/downloads
# Enable 'Add Python to PATH' during installation.
# Download Git from https://git-scm.com/download/win
```

2.3 Cloning & Setting Up dev-mcp

```
# 1. Clone the repository
git clone https://github.com/your-org/dev-mcp.git
cd dev-mcp

# 2. Create a virtual environment
python3 -m venv .venv

# 3. Activate the virtual environment
#   Linux / macOS:
source .venv/bin/activate
#   Windows (PowerShell):
.venv\Scripts\Activate.ps1

# 4. Install dependencies
pip install -e .
# OR using uv (faster):
uv pip install -e .
```

TIP Using uv is recommended — it resolves dependencies significantly faster than pip.

2.4 Verify Installation

```
# Confirm Python path
.venv/bin/python --version

# Run built-in health check
```

```
.venv/bin/python tests/validate_mcp_flow.py
```

3. Connecting Your AI Client

Each AI client needs a JSON configuration snippet telling it where to find the dev-mcp server. The two required values are the **Python command** (full path to .venv Python binary) and the **Server script** (full path to server.py).

IMPORTANT

Replace /home/dev/Desktop/projects/mcp/dev-mcp with the actual path where you cloned dev-mcp.

JSON Configuration Snippet (copy and paste this):

```
{
  "mcpServers": {
    "dev-mcp": {
      "command": "/home/dev/Desktop/projects/mcp/dev-mcp/.venv/bin/python",
      "args": ["/home/dev/Desktop/projects/mcp/dev-mcp/server.py"]
    }
  }
}
```

3.1 Cursor

- Open Cursor and press Ctrl+Shift+P (Cmd+Shift+P on macOS).
- Search for "Open MCP Settings" and select it.
- Paste the JSON snippet above into the MCP configuration file.
- Save the file. Cursor will automatically detect and connect to dev-mcp.

3.2 Claude Desktop

- Open Claude Desktop.
- Go to Settings > Developer > MCP Configuration.
- Click Edit Config to open the JSON file.
- Paste the JSON snippet and save.
- Restart Claude Desktop for changes to take effect.

3.3 Windsurf

- Open Windsurf > Settings (gear icon).
- Navigate to the MCP / Extensions section.
- Add a new MCP server entry with the JSON snippet.
- Save and reload the window.

3.4 Cline / Roo Code

- Inside VS Code, open the Cline or Roo Code extension panel.
- Click the + button to add an MCP server.
- Select Custom Server (stdio) and paste the JSON snippet.
- Click Connect.

3.5 Antigravity CLI

Antigravity CLI is the agentic command-line developer environment from Google DeepMind. It reads local configuration files to spin up dev-mcp as a subprocess automatically, making the AI agent fully project-aware.

- Install Antigravity CLI following the official Google DeepMind documentation.
- In your project root, create or edit the local MCP config file (e.g. `.antigravity/mcp.json`).
- Paste the JSON snippet above into that config file.
- Run your Antigravity agent — it will auto-connect to dev-mcp.

NOTE

Antigravity CLI uses dev-mcp to read project context, evaluate directories, track milestones, and perform architectural checks — all automatically.

4. The Efficiency Workflow Loop

dev-mcp is designed around a **linear sprint lifecycle**. Always follow this order. Do not run ad-hoc commands outside this flow.

Step	Tool	What It Does
1	<code>assess_project_readiness()</code>	Scan workspace for context files. Returns maturity score 0-100%.
2	<code>adopt_existing_project()</code> or <code>bootstrap_project()</code>	Adopt a legacy codebase OR bootstrap a brand-new one.
3	<code>start_session()</code>	Load previous handoff, PRD, and sprint targets.
4	<code>sync_workspace()</code>	Sync physical files and git state with the task backlog.
5	<code>next_best_action()</code>	Get the AI recommended priority task for this sprint.
6	<code>verify_work(task_id=...)</code>	Deep-check code changes for the target task.
7	<code>end_session(session_summary=...)</code>	Save the session summary and generate a handoff log.
8	<code>prepare_commit(session_summary=...)</code>	Analyse modified files and generate commit scopes.
9	<code>commit_changes(message=...)</code>	Stage all files and commit to local git.

4.1 Onboard a New Project (First Time)

```
assess_project_readiness(path=".")
doctor(path=".")

# For new projects:
bootstrap_project(path=".")

# For existing codebases:
adopt_existing_project(path=".")
```

4.2 Daily Development Session

```
start_session(path=".")
next_best_action(path=".")

# After making code changes:
sync_workspace(path=".")
verify_work(task_id="TASK-001", path=".")
verify_outcome(task_id="TASK-001", path=".")
```

4.3 Closing a Session & Committing

```
end_session(session_summary="Implemented auth module and unit tests.", path=".")
prepare_commit(session_summary="Auth module and tests.", path=".")
commit_changes(message="feat(auth): add login flow and unit tests", path=".")
```


5. Architecture & Technology Rules

You can define allowed and forbidden technologies directly in `.project-context.json` under `architecture.rules`. The guardian then scans all source file imports and flags violations.

```
{
  "architecture": {
    "rules": {
      "forbidden_technologies": ["redux", "firebase"],
      "required_technologies": ["typescript"]
    }
  }
}
```

After updating the rules, run:

```
validate_architecture(path=".")
```

HOW IT WORKS

`validate_architecture()` scans all source files for import statements and checks them against your forbidden and required lists. Any violation is reported with the offending file and line number.

Additional Architecture Tools

- `build_dependency_graph()` — Builds a JSON map of all module imports.
- `impact_analysis(file_path="src/auth.py")` — Shows all files transitively affected by changing a target file.

6. Reconnecting After Updates

AI clients cache the MCP server stdio stream. If you update `server.py` or rebuild the virtual environment, the client may show `EOF` or `client closing` errors. Use the table below to reconnect:

Client	How to Reconnect
Cursor	Settings > Features > MCP > Find dev-mcp > click Restart.
Claude Desktop	Fully Quit the application from the OS taskbar/dock, then reopen it.
Windsurf	Press <code>Ctrl+Shift+P</code> > Reload Window, or restart the MCP extension.
Cline / Roo Code	Restart the editor extension panel, or type <code>/mcp restart dev-mcp</code> .
Antigravity CLI	Re-run the CLI command — it re-spawns the server subprocess automatically.

7. Testing Your Server

7.1 End-to-End Validation

After modifying server.py or adding new tools, always run the full lifecycle test:

```
cd /path/to/dev-mcp
.venv/bin/python tests/validate_mcp_flow.py
```

WHAT THIS TESTS

The validation script runs the entire sprint lifecycle from `assess_project_readiness` to `commit_changes` and verifies that every tool responds correctly.

7.2 Adding a New Tool

- Create your tool function in the relevant module (e.g. `brain/my_module.py`).
- Append the import line at the top of `server.py`.
- Re-run the E2E test: `.venv/bin/python tests/validate_mcp_flow.py`
- Reconnect your AI client (see Section 6).

WARNING

Never forget to append your import to `server.py`. A missing import is the most common cause of tool registration failures.

8. Complete Tool Reference (70 Tools)

All 70 tools registered in server.py, organised by module. Most tools accept an optional `path: str = "."` argument pointing to your project root.

8.1 Onboarding & Diagnostics

Module: `brain/project_readiness.py`

Tool	Parameters	Description
<code>assess_project_readiness</code>	<code>path="."</code>	Scans workspace for metadata context files. Returns maturity level (0-5) and score (0-100%).
<code>project_readiness_report</code>	<code>path="."</code>	Compiles a detailed Markdown report of missing files, audits, and recommended actions.
<code>bootstrap_project</code>	<code>path="."</code>	Generates fresh template configs (<code>.project-context.json</code> , <code>.project_brain/</code>) for new projects.
<code>adopt_existing_project</code>	<code>path="."</code>	Scans existing dirs and configs, parses README.md and dependencies, maps to PM contexts.
<code>doctor</code>	<code>path="."</code>	Performs a 7-layer health check (server, git, context JSON, PRD, requirements, task backlog).
<code>reset_project_brain</code>	<code>path="."</code>	Deletes context stores and <code>.project_brain/</code> folders. Source code and git logs remain untouched.
<code>archive_project_brain</code>	<code>path="."</code>	Packages active Project Brain into a timestamped folder under <code>.project_brain_archive/</code> .

8.2 Session & Sprint Orchestration

Module: `brain/orchestrator.py`

Tool	Parameters	Description
<code>generate_project_brain</code>	<code>path="."</code>	Produces an aggregated dashboard JSON summarising tasks, phase metrics, and commit records.
<code>next_best_action</code>	<code>path="."</code>	Calculates the next development task based on phase, active priority, and unmet dependencies.
<code>start_session</code>	<code>path="."</code>	Initialises a sprint session and prints a Markdown brief of targets, PRDs, and last handoff.
<code>end_session</code>	<code>session_summary,</code> <code>known_issues="", path="."</code>	Ends the sprint, saves summaries, and logs handoffs without committing code.

Tool	Parameters	Description
<code>prepare_commit</code>	<code>session_summary, path="."</code>	Analyses modified/untracked files and recommends conventional commit scopes.
<code>commit_changes</code>	<code>message, path="."</code>	Stages all modified files and commits them to the local git repository.

8.3 Structured Task Management

Module: `brain/todo_engine.py`

Tool	Parameters	Description
<code>create_task</code>	<code>title, description, phase="", priority="medium", depends_on="", path="."</code>	Appends a task with priority, phase, and dependencies to the active backlog.
<code>update_task</code>	<code>task_id, title="", description="", phase="", status="", priority="", depends_on="", path="."</code>	Modifies properties of an existing task.
<code>complete_task</code>	<code>task_id, notes="", path="."</code>	Marks a task as completed and moves it to completed history.
<code>next_task</code>	<code>path="."</code>	Returns the highest-priority task that is ready to work on.
<code>blocked_tasks</code>	<code>path="."</code>	Lists all tasks blocked by unfinished dependencies.
<code>list_tasks</code>	<code>phase="", status="", path="."</code>	Returns all tasks, optionally filtered by phase or status.

8.4 Phase & Milestone Scoping

Module: `brain/phase_manager.py`

Tool	Parameters	Description
<code>get_current_phase</code>	<code>path="."</code>	Prints the active phase, its description, and tasks.
<code>set_phase</code>	<code>phase_name, description="", path="."</code>	Sets the active phase. Only tasks in this phase are in scope.
<code>complete_phase</code>	<code>notes="", path="."</code>	Concludes the active phase and logs the decision.
<code>next_phase</code>	<code>phase_name, description="", path="."</code>	Advances the project from the current phase to a new phase.
<code>list_phases</code>	<code>path="."</code>	Shows all past, present, and pending phases.
<code>add_milestone</code>	<code>name, target_date="", features="", path="."</code>	Creates a milestone with a target date and comma-separated feature list.
<code>list_milestones</code>	<code>path="."</code>	Displays all release milestones.

8.5 PRD Management

Module: *brain/prd_manager.py*

Tool	Parameters	Description
<code>get_prd</code>	<code>document="PRD", path="."</code>	Reads a stored PRD file (VISION, SCOPE, REQUIREMENTS, or PRD).
<code>update_prd</code>	<code>document, content, path="."</code>	Creates or overwrites a target PRD document.
<code>summarize_prd</code>	<code>path="."</code>	Extracts bullet summaries from all PRD documents.
<code>compare_work_to_prd</code>	<code>path="."</code>	Compares active tasks against requirements to trace scope gaps.

8.6 Requirements & Outcome Auditing

Module: *brain/requirements.py*

Tool	Parameters	Description
<code>extract_requirements</code>	<code>path="."</code>	Parses PRD markdown docs to extract structured requirement keys into requirements.json.
<code>verify_requirements</code>	<code>feature_name="", path="."</code>	Scans files for keywords to audit requirements implementation status.
<code>verify_outcome</code>	<code>task_id, path="."</code>	Runs UI and API checks on a target task to verify business objectives.

8.7 Workspace Sync

Module: *brain/sync_engine.py*

Tool	Parameters	Description
<code>sync_workspace</code>	<code>path="."</code>	Compares physical files, modified lines, branch names, and git logs to detect completed work.
<code>verify_work</code>	<code>task_id, path="."</code>	Performs deep code checks (tests, git commits, guardian rules) to generate verification dossiers.
<code>project_dashboard</code>	<code>path="."</code>	Compiles a developer console showing phases, task tallies, architecture details, and health scores.

8.8 AI Handoffs

Module: *brain/handoff_engine.py*

Tool	Parameters	Description
<code>generate_handoff</code>	<code>session_summary="", known_issues="", path="."</code>	Records metrics, sprint goals, and active tasks into markdown session logs.

Tool	Parameters	Description
get_last_handoff	path="."	Loads the latest session handoff markdown file.
list_handoffs	path="."	Lists all past session logs with dates.

8.9 Architecture & Dependencies

Modules: *brain/guardian.py*, *brain/dependency.py*

Tool	Parameters	Description
validate_architecture	path="."	Audits imports against allowed/forbidden technologies defined in context rules.
build_dependency_graph	path="."	Constructs a JSON representation of module import maps.
impact_analysis	file_path, path="."	Recursively maps all files affected if a target file is modified (transitive impact).

8.10 Project Context CRUD

Module: *tools/context_mgmt.py*

Tool	Parameters	Description
init_context	path="."	Detects tech stacks and initialises .project-context.json.
get_context	dot_path="", path="."	Queries context values using dot notation (e.g. tech_stack.languages).
update_context	dot_path, value, path="."	Sets a dot-path target value in .project-context.json.
add_to_context_list	dot_path, item, path="."	Appends string or JSON items to context lists.
add_key_file	file_path, purpose, details="", root_path="."	Registers files and their development purposes.
add_task	task_id, description, status="pending", branch="", notes="", path="."	Registers active tasks directly into the context.
add_decision	decision, reason, alternatives="", path="."	Appends Architecture Decision Records (ADRs) to the context.
export_context_markdown	path="."	Converts context to standard formatted Markdown files.
remove_context_item	dot_path, index, path="."	Removes list items by zero-based index.
reset_context	path="."	Wipes active context structures.
project_info	path="."	Prints basic directory file statistics.

8.11 File Sandbox Operations

Module: `tools/file_ops.py`

Tool	Parameters	Description
<code>read_file</code>	<code>path</code>	Reads file content in authorised folders.
<code>write_file</code>	<code>path</code> , <code>content</code>	Writes content, creating directory hierarchies if needed.
<code>edit_file</code>	<code>path</code> , <code>old_string</code> , <code>new_string</code>	Performs case-sensitive string replacements in a file.
<code>delete_path</code>	<code>path</code>	Removes files or empty folders.
<code>delete_recursive</code>	<code>path</code>	Recursively removes an entire directory tree.
<code>list_directory</code>	<code>path</code>	Lists folder directories and file sizes.
<code>glob_files</code>	<code>pattern</code> , <code>base_path="."</code>	Searches for files matching glob rules (e.g. <code>**/*.ts</code>).
<code>create_directory</code>	<code>path</code>	Creates a new directory.

8.12 Terminal & Git Core Helpers

Modules: `tools/terminal.py`, `tools/search.py`, `tools/git_ops.py`

Tool	Parameters	Description
<code>run_command</code>	<code>command</code> , <code>timeout_seconds=60</code> , <code>workdir=None</code>	Executes shell commands in sub-processes with timeout safeguards.
<code>search_code</code>	<code>pattern</code> , <code>path="."</code> , <code>glob_filter=None</code> , <code>max_results=50</code>	Codebase search using Ripgrep or Python fallback patterns.
<code>git_status</code>	<code>repo_path="."</code>	Queries working git directory status.
<code>git_diff</code>	<code>repo_path="."</code> , <code>staged=False</code>	Exposes modified staged/unstaged branch differences.
<code>git_log</code>	<code>repo_path="."</code> , <code>max_count=10</code>	Returns recent commit logs.
<code>git_branch</code>	<code>repo_path="."</code>	Lists all local branch names.
<code>git_add</code>	<code>repo_path="."</code> , <code>files="."</code>	Stages target files for commit.
<code>git_commit</code>	<code>repo_path="."</code> , <code>message="update"</code>	Stages and commits modifications.
<code>git_create_branch</code>	<code>repo_path="."</code> , <code>branch_name=""</code>	Creates and checks out a target git branch.

■ dev-mcp | AI Engineering Operating System

v0.1.0 | Author: Pranjal Yadav | Model Context Protocol (MCP)